

OpenPI 真机控制时序与同步假设分析

Codex

2026 年 5 月 31 日

摘要

本文分析目标仓库中 OpenPI 真机部署路径的推理、动作排程与底层执行机制，并将其与《The Speedup Paradox: Rethinking Inference Optimization in Embodied Tasks》中的静态同步假设进行对照。结论是：当前真机路径即使关闭显式异步推理，也不是论文中理想化的严格同步执行，而是带有真实时间戳、过期动作丢弃、动作截断和底层插值控制的时间戳排程式同步。因此，单纯放大推理延迟 T_{inf} 会改变实际执行的动作索引、有效动作数和低层控制器接受的 waypoint，从而可能显著恶化控制质量。LIBERO 这类典型静态模拟器的默认评测机制则更接近论文同步假设，因为仿真状态通常只在 `env.step` 时推进，阻塞式推理不会被真实 wall-clock 时间转化为动作过期。

1 问题背景

论文的静态任务实验把推理延迟视作一个可以独立改变的变量。换言之，在观测相同、模型相同、轻量化设置相同的条件下，如果仅改变 T_{inf} ，理论上改变的是 $\rho = T_{\text{inf}}/T_{\text{act}}$ ，而不是动作序列本身。这个假设适合推导 chunk 级时间收益，也适合在静态模拟器中隔离硬件速度的影响。

用户在 RTX 4070 上对 pick-and-place 任务进行去噪步数实验：10 步、5 步和 1 步去噪使 T_{chunk} 只有小幅下降，但动作质量随去噪步数减少而变差，导致完成任务所需 chunk 数或动作数 N 明显上升，最终 T_{task} 不降反升。按照论文理论，如果硬件算力变差、 ρ 升高，则减少去噪步数带来的 T_{chunk} 降幅应当更显著，可能出现某个中间步数的最优点，甚至最优点向小步数方向移动。

实际放大 T_{inf} 后，真机控制质量却严重下降，并出现过冲。这说明真实部署中“改变推理延迟”并不只是改变 ρ 。关键原因在于 OpenPI 真机路径把模型输出映射到真实时间戳上的 future actions，并在推理完成后根据当前 wall-clock 时间丢弃已经过期的动作。于是，延迟变大将改变被执行的 action index、有效执行数量以及动作相对底层控制器的时间裕量。

2 论文中的理想同步模型

论文第 3 节给出的同步基线可写为

$$T_{\text{chunk}} = T_{\text{inf}} + nT_{\text{act}}, \quad (1)$$

其中 T_{inf} 是一次策略推理的耗时, n 是一个 chunk 中实际执行的动作数, T_{act} 是单个动作的执行时间。任务总时间写作

$$T_{\text{task}} = NT_{\text{chunk}}, \quad (2)$$

而硬件和控制频率的相对关系由

$$\rho = \frac{T_{\text{inf}}}{T_{\text{act}}} \quad (3)$$

刻画。

这一模型隐含几个强假设。第一, 推理阶段和执行阶段串行排列; 推理完成后执行固定数量的动作。第二, 每轮实际执行的 n 不随 T_{inf} 改变。第三, 模型输出动作的内容和被执行的动作索引不因推理延迟而改变。第四, 对于静态任务, 推理期间环境状态不会发生与任务成功相关的变化。论文的 limitations 也明确承认其分析抽象掉底层执行细节, 并假设仅改变 inference latency 不会改变轨迹。

如果一个系统严格满足这些假设, 那么放大 T_{inf} 的作用主要体现在 T_{chunk} 上; 除非模型质量或轻量化程度改变, N 不应因为硬件延迟本身发生系统性变化。当前真机框架的问题正在于: 代码中存在多处把 T_{inf} 转化为动作丢弃、动作重索引和底层排程裕量变化的机制。

3 OpenPI 真机路径的执行语义

3.1 显式异步推理不是唯一的异步来源

在 `deploy/inference_real.py` 中, 默认参数包含 `frequency=10.0`、`steps_per_inference=6`、`action_exec_latency=0.01`、`async_inference=False` 和 `inference_overlap_steps=0`。当 `async_inference=False` 时, 主循环确实会阻塞等待 `policy_infer(policy_obs)` 返回, 因此不存在显式后台推理线程。但是, 这不等价于论文的严格同步。推理返回后, 动作不是无条件从第 0 个开始执行, 而是先被映射到以观测时间为起点的时间戳序列, 再按当前时间筛掉过期动作。

这种机制可抽象为

$$t_k = t_{\text{obs}} + k\Delta t, \quad \Delta t = \frac{1}{f}, \quad (4)$$

并选择满足

$$t_k > t_{\text{now}} + \delta_{\text{exec}} \quad (5)$$

的动作。若 T_{inf} 增大, 则 t_{now} 后移, 最小可执行索引 k_0 随之增大。于是模型输出的前若干动作会被直接跳过。实际执行的动作数近似为

$$n_{\text{client}} = \min(n_{\text{cfg}}, H - k_0), \quad (6)$$

其中 H 是模型 action horizon, n_{cfg} 对应 `steps_per_inference`。因此, 即使配置中的 n_{cfg} 不变, 有效执行的首动作索引和动作集合也会随 T_{inf} 改变。

3.2 真实控制器还会再次处理时间戳

动作通过 `env.exec_actions(..., compensate_latency=True)` 下发到 UMI 环境。`bimanual_umi_env.py` 会再次根据接收时间过滤 future timestamps, 并在补偿延迟时从目标时间中减去 `robot_action_latency` 和 `gripper_action_latency`。配置文件 `deploy/configs/umi_ur5e_wsg50.yaml` 中这两个延迟均为 0.1 秒, 而 `inference_real.py` 默认的 `action_exec_latency` 只有 0.01 秒。若客户端认为某动作仍在未来, 但底层补偿后 waypoint 时间已经接近当前时间甚至落在过去, RTDE 插值控制器可能无法按预期接收该 waypoint。

底层 UR 控制器不是在每个策略动作处停顿等待下一次推理, 而是通过 `rtde_interpolation_controller` 中的 500Hz `servoL` 循环持续运行。策略侧发送的是未来 waypoint; 控制器侧将 wall-clock 时间转换为 monotonic 时间并排入插值轨迹。`pose_trajectory_interpolator.py` 中对于时间不大于当前时刻的 waypoint 直接返回原轨迹, 不会产生新的有效段。这进一步说明, 放大 T_{inf} 可能改变的不只是 chunk 时间, 还包括底层轨迹是否真正被更新。

3.3 延迟仿真会改变动作排程而非只改变等待时间

`deploy/latency_simulation.py` 的 `scaled_infer` 先执行真实推理, 再按照耗时比例额外 sleep。也就是说, 模型输出本身不变, 但推理返回时的 wall-clock 时间改变。对于论文模型, 这相当于只改变 T_{inf} 。对于当前真机代码, 这会让 `_future_action_schedule` 看到更晚的 `time.time()`, 从而丢弃更多前缀动作, 并改变实际执行的 action index。这个差异正好解释了为什么“控制模型轻量化程度一定, 只放大推理延迟”仍然会导致严重控制质量下降。

4 代码证据

表 1 汇总了支撑上述判断的关键代码位置。行号对应本次分析时工作区中的文件内容。

表 1: OpenPI 真机路径中的时序机制证据

代码位置	机制	对论文同步假设的影响
<code>deploy/inference_real.py</code> 33-40	默认控制频率为 10Hz, <code>steps_per_inference</code> 默认为 6, <code>action_exec_latency</code> 默认为 0.01 秒; 显式异步推理默认关闭。	关闭 <code>async_inference</code> 只说明没有后台策略线程, 不说明执行语义等于论文中的严格同步。

代码位置	机制	对论文同步假设的影响
deploy/inference_real.py 160-178	future action schedule 以 obs_timestamp 为起点给整个 action chunk 生成时间戳，并保留晚于 time.time()+action_exec_latency 的动作。	T_{inf} 增大会让更多前缀动作被判定为过期，实际执行的动作索引随延迟改变。
deploy/inference_real.py 181-192	将 future actions 截断到 max_scheduled_actions 或 steps_per_inference。	配置中的 n 是上限，不是实际保证执行的动作数量。
deploy/inference_real.py 417-421	同步路径阻塞调用 policy_infer(policy_obs)，并记录 infer_ms 和 obs_timestamp。	策略推理是同步等待的，但推理完成后的动作选择仍依赖真实时间。
deploy/inference_real.py 424-425	动作下发使用 env.exec_actions(..., compensate_latency=True)。	客户端选择的 future action 还会进入底层延迟补偿路径。
deploy/inference_real.py 482-489	循环按配置中的计划步数推进 step_idx，而不是按底层实际接受 waypoint 数推进。	实验中观测到的有效 n 可能和循环索引中的 n 不一致。
deploy/inference_real.py 494-500	异步模式通过 overlap steps 在动作执行期间提前发起下一轮推理。	若开启该模式，则系统进一步偏离论文的串行同步基线。
deploy/latency_simulation.py 61-70	scaled_infer 在真实推理后按比例额外 sleep，并返回相同动作。	它看似只改变 T_{inf} ，但在真机代码中会改变 future action 过滤结果。
deploy/async_inference.py 42-49	异步 worker 使用单个后台线程提交策略推理任务。	这是显式异步推理来源；不过即使关闭它，时间戳排程仍然存在。
deploy/umi/real_world/bimanual.py 420-421	客户端 future action 到达环境后仍可能被二次丢弃。	客户端 future action 到达环境后仍可能被二次丢弃。
deploy/umi/real_world/bimanual.py 441-446	排程时从目标时间减去机器人和夹爪动作延迟。	若客户端 guard 过小，补偿后的 waypoint 可能落到过近或过去。

代码位置	机制	对论文同步假设的影响
deploy/configs/umi_user_cfg.yaml	robot_action_latency=0.1 且 gripper_action_latency=0.1。	底层补偿量显著大于客户端默认 action_exec_latency=0.01。
deploy/umi/real_world/rtdr_chunk_server.py	删除 500Hz 循环并持续执行插值轨迹。	底层块不是连续控制系统，不是每个 chunk 后停住等待推理的离散系统。
deploy/umi/real_world/rtdr_chunk_client.py	目标时间被转换为 waypoint 间后排入插值器。	waypoint 可达性取决于 waypoint 到达控制器时是否仍有足够未来时间。
deploy/umi/common/pose_tracking.py	若 waypoint 时间不晚于当前时间，则返回原轨迹。	waypoint 到达的动作不会形成新的有效轨迹段。
src/openpi/training/config.py	配置 action_horizon 为 30。	模型输出长度 $H = 30$ ，但真机每轮通常只执行其中一小段。

5 遥测结果与机制解释

现有实验摘要位于 data/umi_step_p3_trials_analysis/plots/tisd_chunk_metrics_trimmed_1_7_10 表 2 显示，在 4070 上减少去噪步数并未显著降低 T_{chunk} ，但显著增加了 N ，因此 T_{task} 上升。

表 2: 去噪步数实验的初步摘要				
去噪步数	T_{chunk} 均值/秒	N 均值	T_{task} 估计/秒	
1	0.7912	57.4	45.42	
7	0.7873	49.2	38.74	
10	0.7894	47.2	37.26	

另外，遥测中常见 model_action_count=30，而 scheduled_action_count 约为 8，dropped_action_count 约为 22，首个被排程动作的索引经常为 4 或 5。这与代码机制一致：模型生成了完整 horizon，但客户端只挑出仍在未来的一段动作，再截断后下发。若人为放大 T_{inf} ，首个可执行索引会继续后移；在 pick-and-place 这类需要精细接近和闭合的任务中，跳过前缀动作会造成动作相位错位、夹爪闭合时机偏移和末端过冲。

因此，在当前真机部署路径中，观测到的 N 上升不应只归因于去噪步数改变带来的模型动作质量下降。额外的推理延迟还可能通过 action dropping 和 waypoint 过期机制降低控制质量。

换言之，真实系统中的 N 可以写作

$$N = N(q_{\text{model}}, k_0, n_{\text{client}}, n_{\text{accepted}}, \epsilon_{\text{schedule}}), \quad (7)$$

其中 q_{model} 表示模型动作质量， k_0 表示实际起始动作索引， n_{client} 表示客户端下发动作数， n_{accepted} 表示底层控制器实际接受的 waypoint 数， $\epsilon_{\text{schedule}}$ 表示排程误差。论文静态模型主要把 N 看作模型质量和任务难度的结果，而当前真机路径使 N 同时依赖 wall-clock 时序。

6 LIBERO 与论文假设的距离

典型 LIBERO 评测机制更接近论文中的同步假设。以 `examples/libero/main.py` 为例，默认 `replan_steps=5`。程序只在 `action_plan` 为空时调用策略生成 `action chunk`，然后取 `action_chunk[:replan_steps]` 放入队列。每一轮从队列中弹出一个动作并调用 `env.step(action.tolist())`。在这个过程中，仿真状态只随 `env.step` 推进；阻塞等待策略推理不会让模拟世界按真实时间继续运动，也不会让旧动作因 wall-clock 时间过期。

因此，LIBERO 默认机制与论文假设的关系可以概括如下。它不是完全等同于论文，因为实际 n 是 `replan_steps`，不一定等于模型的 action horizon；而且不同评测脚本可能加入自己的缓存或并行化策略。但相对于真机 OpenPI，LIBERO 少了真实时间戳过滤、底层控制器延迟补偿、网络传输、500Hz 插值控制器和 waypoint 过期逻辑。若在 LIBERO 中人为 `sleep` 以放大 T_{inf} ，通常只会增加每次 `replan` 的等待时间，而不会自动跳过 `action chunk` 的前缀动作。这正是它更接近论文干净同步假设的核心原因。

7 为什么真机不默认采用严格同步

真机和 LIBERO/论文假设拉开距离并不是任意设计，而是由真实机器人控制条件决定的。第一，真实机械臂必须持续接收平滑轨迹或伺服目标。若策略推理期间机器人完全停等，末端会频繁进入 `stop-and-go` 状态，导致速度不连续、接触不稳定，并可能触发底层控制保护。第二，相机、网络、策略服务器和机器人控制器之间存在不可忽略且波动的延迟，系统必须用未来时间戳和 `latency compensation` 把动作安排到控制器可执行的未来。第三，真实世界在推理期间不会冻结；物体、夹爪、末端和传感器状态都在持续变化。第四，底层 RTDE 控制器以高频连续插值执行，策略频率只有约 10Hz，两者天然需要一个时序桥接层。

因此，真机系统更关心连续、平滑、可抢占和可容忍延迟抖动的控制，而论文静态同步假设更关心隔离 T_{inf} 、 T_{act} 、 n 和 N 的理论关系。严格同步可以作为受控实验模式，但不一定是生产式真机控制的默认值。

8 所谓同步推理平台中的异步层

更一般地说，不能简单断言“所有号称同步推理的真机平台都在做异步推理”。更准确的区分是：许多平台在策略 API 层确实是同步阻塞的，即用户代码调用 `infer()` 或发送一次请求后等待返回；但在机器人系统层，它们几乎总会保留某种异步通信、轨迹缓冲、时间戳排程或高

频控制环。因此，“同步推理”通常只描述 policy-call 层面的阻塞语义，不等价于论文中“推理期间物理世界冻结，推理返回后从 action chunk 第 0 个动作开始严格执行固定 n 个动作”的理想同步。

这种差异在主流机器人中很常见。ROS 2 将 service 描述为短任务的同步 request/response 交互，而 action 则面向长时间任务，带 feedback、result 和 cancel/preempt 语义；官方文档也明确指出 action server 在接收 goal 后执行任务并持续反馈，action 适用于可能耗时较长的操作¹。因此，即使 action client 选择阻塞等待 result，server 侧运动执行仍是一个独立的长过程，而不是论文式的单线程停等。Universal Robots 的 RTDE 文档也把 RTDE 定义为一种让外部应用通过 TCP/IP 与 UR controller 同步、同时不破坏控制器实时属性的接口，并包含 setup 与 synchronization loop 两个阶段²。这说明外部策略程序和机器人控制器之间本来就是两个时钟域。Franka 的 libfranka 文档同样说明 `franka::Robot::control` 会以 1kHz 频率执行用户回调，机器人状态也以 1kHz 提供³。高层策略是否阻塞等待推理，并不会消除这个底层实时控制循环。

因此，评估一个所谓“同步推理”的真机平台时，应该至少分清四层语义。第一是策略调用是否同步：`infer()` 是否阻塞到模型返回。第二是动作下发是否同步：发送 trajectory 或 goal 后是否等待确认或结果。第三是控制器是否同步：底层是否仍在固定频率上插值、伺服或跟踪上一段轨迹。第四才是论文式严格同步：推理期间不执行旧轨迹、不按真实时间丢弃动作、不重排 action index，每轮都从第 0 个动作开始执行固定 n 个动作。主流真机平台通常最多满足前两层，而第三层几乎必然是独立运行的；第四层更像受控实验模式，需要显式实现和记录，而不是生产系统默认语义。

9 当前实现：论文式严格同步与严格异步

若目标是复现实验中“只改变 T_{inf} ，不改变动作索引和有效 n ”的理想同步或严格异步，应使用专门的实验模式，而不是复用当前 `practical --async-inference` 路径。当前分支已经在 `deploy/inference_real.py` 中加入了两个互斥模式：`--strict-sync` 和 `--strict-async`。

9.1 严格同步

`--strict-sync` 的语义是：观察当前状态；阻塞推理；推理结束后从 action chunk 的第 0 个动作开始取固定数量 n ；以推理完成后的当前时间为新的执行起点，将这 n 个动作重锚定到未来；等待这 n 个动作执行完，再进入下一轮观测和推理。该模式绕过 `_future_action_schedule` 对 `obs_timestamp` 的依赖。时间戳为

$$t_k = t_{\text{return}} + g + k\Delta t, \quad k = 0, \dots, n-1, \quad (8)$$

¹<https://docs.ros.org/en/rolling/Concepts/Basic/About-Actions.html>

²https://www.universal-robots.com/manuals/EN/HTML/SW10_10/Content/Prod-RTDE/Real_Time_Data_Exchange_RTDE.htm

³<https://support.franka.de/docs/libfranka.html>

其中 t_{return} 是推理返回时间, g 对应 `--strict-sync-start-delay`, $\Delta t = 1/f$ 。若仍启用 `compensate_latency=True`, 则 g 必须至少覆盖 robot/gripper latency 和通信抖动; 若希望最贴近论文假设, 则可以在严格实验模式中关闭底层补偿, 并选择一个足够保守的固定 future start。关键点不是 guard 的具体数值, 而是每次都执行同一组动作索引 $0, \dots, n-1$, 且实际执行数量固定。

9.2 严格异步

`--strict-async` 对应论文中的固定 overlap 异步模型。设每个 chunk 固定执行 n 个动作, 固定 overlap 步数为 n' , 则下一轮推理在当前 chunk 执行到 $n-n'$ 个动作后启动。若当前 chunk 的最后一个动作时间戳为 t_{n-1} , 则下一 chunk 的计划起点为

$$t_{\text{target}} = t_{n-1} + \Delta t, \quad (9)$$

而下一轮推理启动时间为

$$t_{\text{launch}} = t_{\text{target}} - n' \Delta t. \quad (10)$$

这样, 模型推理和机械执行的理论 overlap 窗口固定为 $n' \Delta t$ 。若下一轮推理在 t_{target} 前完成, 则下一 chunk 贴着计划边界开始; 若推理晚于边界完成, 则下一 chunk 起点顺延到

$$\max(t_{\text{target}}, t_{\text{return}} + g). \quad (11)$$

在这两种情况下, 下一 chunk 仍然从 action index 0 开始取固定数量动作, 不会因为 wall-clock 过期而跳过 chunk 前缀。

该模式通过 `--strict-async-overlap-steps` 设置 n' , 并要求 $0 < n' \leq n$ 。它与 `--async-inference` 和 `--strict-sync` 互斥, 因为三者代表不同实验语义: practical async 追求真机连续性, strict sync 追求完全串行可控, strict async 追求固定 overlap 的论文式异步。

9.3 遥测检查

实验记录现在包含 `schedule_mode`、`scheduled_action_indices`、`first_scheduled_action_index`、`last_scheduled_action_index` 和 `prefix_dropped_action_count`。严格异步还会记录 `strict_async_overlap_steps`、`strict_async_overlap_window_ms`、`strict_async_target_start_timestamp`、`strict_async_launch_timestamp` 和 `strict_async_boundary_wait_ms`。只有当 `first_scheduled_action_index` 和 `prefix_dropped_action_count=0` 且固定动作数稳定时, 该轮才真正满足论文式 strict sync/strict async 的动作索引条件。

10 结论

当前 OpenPI 真机框架的同步模式可以称为“阻塞推理 + 时间戳排程”, 不能称为论文中的严格同步。它的实际 ρ 不是简单的 $T_{\text{inf}}/T_{\text{act}}$, 因为 T_{inf} 同时影响 action dropping、动作起始索引、底层 waypoint 裕量和有效 n 。这解释了为什么在放大 T_{inf} 时会出现控制质量严重下降甚至过冲。

LIBERO 这类静态模拟器默认更接近论文假设，因为推理期间仿真状态通常不会按真实时间推进，动作也不会因 wall-clock 延迟过期。若要在真机上验证论文理论，应使用当前新增的 `--strict-sync` 或 `--strict-async` 实验模式，把动作 chunk 在推理返回后或固定 overlap 边界处重新锚定到未来，并保证每轮执行固定动作索引和固定数量。所谓“同步推理”本身不足以保证这一点，因为它往往只约束策略调用层，而不约束底层控制器、通信队列、trajectory buffer 和 timestamp scheduler。否则，实验测到的是模型质量、硬件延迟、时间戳排程和底层控制器共同作用后的系统表现，而不是论文中单独改变 ρ 或固定 n' overlap 的理想化结果。